Exploring Mechanistic Interpretability: A Greek Case Study on Gemma 2B

Aristeidis Pagonopoulos, Thomas Nektarios Saltzis, Nick Tzavidas
Department of Informatics, Aristotle University of Thessaloniki

***Abstract*—The rapid evolution of neural architectures—from CNNs and RNNs to Transformers—has achieved unprecedented performance across domains, yet their internal mechanisms remain opaque, particularly in large language models (LLMs). While existing interpretability techniques provide partial insights, they struggle with LLMs' depth, attention complexity, and high-dimensional representations. This work demonstrates how sparse autoencoders (SAEs) can decompose LLM activations into interpretable feature dictionaries, uncovering human-aligned concepts like linguistic themes and emotional tones. Using Gemma-2B and a pretrained SAE from GemmaScope, we identify a Greek language feature in the 23rd residual stream layer and use it to steer the baseline Gemma-2B model towards contextually greek related outputs.**

***Index Terms*—Mechanistic Interpretability, Sparse Auto Encoders, Large Language Models**

I. Introduction

The evolution of neural network architectures has witnessed remarkable advancement, progressing from simple feedforward designs through convolutional neural networks (CNNs) and recurrent neural networks (RNNs) to the current era of Transformer-based models. While these architectural innovations have achieved unprecedented performance across computer vision [1], natural language processing [2], and numerous other domains [3], the internal mechanisms driving their success remain opaque.

Understanding why a CNN classifies a particular image or predicting what drives an RNN's sequence generation continues to challenge researchers. Although existing interpretability research has developed valuable tools—including saliency mapping and occlusion studies for CNNs, gate activation analysis for RNNs, and attention pattern visualization techniques—these approaches have yet to converge into a comprehensive mechanistic framework for understanding neural network behavior.

The interpretability challenges become particularly difficult in large language models (LLMs), whose extensive depth, numerous attention mechanisms, and high-dimensional hidden representations render traditional explanation techniques inadequate. The interpretability challenge is further increased by LLMs' effort to model the full richness of human language—encompassing syntactic structure, semantic relationships, and cultural context—resulting in an high dimensional latent feature space that must be disentangled. Contemporary research has explored neuron-level analysis, circuit decomposition, and linear-representation studies, but these methods still struggle to scale and generalize. An emerging solution involves the application of sparse autoencoders (SAEs) to LLM activation patterns [4]. Through sparsity constraints, SAEs can decompose complex activations into interpretable feature dictionaries, where each discovered feature represents a meaningful computational direction within the model's representational space.

These features are discovered in an unsupervised manner and can represent a wide variety of concepts, such as references to animals, sarcastic tone, or aggressive remarks. Once identified, we can steer the model's behavior by selectively amplifying or suppressing the neurons tied to each feature. This targeted approach not only yields fine-grained adjustments but also gives researchers stronger control—making harmful outputs far less likely by directly shaping the model's internal representations.

In our project we used Gemma-2B to investigate this behavior and to find a feature related to greek language. We used a pretrained sparse autoencoder (SAE) from the GemmaScope analysis tool [5], trained on the 23rd layer of the residual stream. Having identified the feature vector, we validated our findings against Neuronpedia [6] and succesfuly steered the model to generate contextually greek related outputs 2.

II. Related work

A. Large Language Models

As noted in the introduction, there has been rapid development of deep neural networks in the past

decade, having a big impact on Natural Language Processing (NLP) tasks. The state of the art (SOTA) for most NLP tasks has been sequence-to-sequence (seq2seq) models, having Recurrent Neural Networks (RNN) at their core. More advanced RNNs, such as Long Short-Term Memory (LSTMs) [7] and Gated Recurrent Unit (GRU) [8], along with more and more such layers stacked together, have enabled better and better performance in NLP tasks.

These architectures, however, lack scalability and parallelization. Since 2015, there has been a paradigm shift towards attention-based methods. In 2015, [9] introduced additive attention, an auxiliary single-layer feedforward network to assign weights to different tokens as part of next token prediction process, essentially stimulating it to pay more attention to the most important tokens of the input. In 2017, [10] introduced the transformer architecture, which alleviates the bottlenecks introduced by RNNs and revolutionized fields like NLP and Computer Vision (CV), giving rise to modern applications, such as Large Language Models.

Since the emergence of the original transformer architecture [10], many new transformer architectures have been developed, such as the decoder-only Generative Pretraining (GPT) [11]. In the years that followed these breakthroughs, the language models scaled massively. GPT-2, launched in 2019, has about 1.5 billion parameters, while modern SOTA open-source Large Language Models (LLMs) have hundreds of parameters. One such SOTA LLM, Deepseek V3 [12], has 671 billion parameters.

In our work, in order to balance our limited computational resources and model usability, we opt to work with a medium-sized LLM, Gemma 2-2B [13], with 2 billion parameters.

B. Interpretability in Large Language Models

Although Large Language Models (LLMs) have demonstrated growing effectiveness across a wide array of tasks, they remain largely "black boxes" whose internal decision-making processes are opaque [14]. Mechanistic interpretability is an emerging field of research that aims to reverse-engineer neural networks by systematically mapping how distinct neuron clusters, attention heads and circuit motifs contribute to model behavior.

Ideally, each neuron in an LLM would encode a single, human-understandable feature, such as "capital letters" or "names of Greek presidents." In practice, however, the imbalance between parameter count and the number of conceivable features, forces many neurons to become polysemantic, meaning they fire for multiple, semantically unrelated concepts (for example, both Greek letters and names beginning with "A") [15]. Polysemanticity is intimately linked to superposition, a hypothesized phenomenon where a neural network represents more features than it has neurons by assigning each feature to a linear combination of neurons [16]. For that reason, mapping individual neurons—or groups of them—to human-readable features becomes almost intractable.

One way to effectively tackle this problem is to impose sparsity, by applying techniques like L_1 regularization, we constrain each neuron's response to just a handful of features. Therefore, by training a sparse autoencoder (SAE) on an LLM's activation vectors, we can uncover an unsupervised dictionary of latent features. Imposing a sparsity penalty on the hidden layer forces each activation vector x to be reconstructed as $x \approx W_d \cdot h$ where W_d is the learned feature-dictionary and h is a sparse code with only a few nonzero entries. Consequently, any single neuron's activation can be expressed as a linear combination of just a handful of those learned features. In addition, just as neurons' activations can be expressed as a linear combination of learned features, features themselves can be represented as linear combinations of neurons. This perspective treats each neuron as a directional basis in activation space, meaning that features emerge as specific weighted combinations of these neuronal directions.

Since features in a Sparse Autoencoder (SAE) are discovered in an unsupervised manner, their correspondence to specific concepts is not immediately known. To label these features, we can use either manual human analysis or an automated approach. In manual analysis, we examine a feature's activations across various text inputs to infer its meaning 1. However, given the large number of features, automation is often necessary. A large language model, such as GPT-4, can generate explanations for features by analyzing examples of tokens that strongly activate them. These explanations can then be leveraged to predict feature activations on previously unseen tokens, improving interpretability in a scalable way.

C. Autoencoder Architectures

Autoencoder (AE) is a special case of encoder-decoder architectures, where both the input and the target output are the same vector. AEs were introduced by [17], as a way learn neural networks

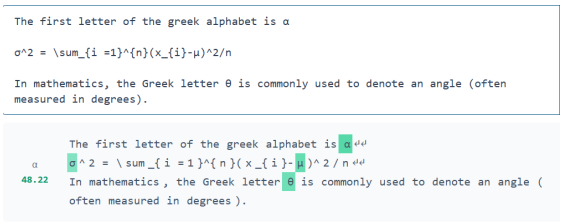


Fig. 1: Activation of the "greek symbols" feature in Gemma-2-2B on sample text, as extracted by the Matryoshka SAE (source: Neuronpedia).

using backpropagation in an unsupervised manner, by having both as inputs and outputs of the network the same set of data.

AEs can be used to compress the input using the encoder portion of the network, scaling it down to a small-dimensional choke point, called the latent space and then decompress this latent vector back to the original input using the decoder part of the network [18]. Essentially, the latent vectors can be thought of as dimensionally-reduced representations of the inputs (similar to the PCA outputs) and can be used to extract meaningful, compressed features from a large input vector that are sufficient to reconstruct the input to a certain degree.

According to [18], the optimization objective of an AE can be a closeness measure between the output of the AE and the original data. In case of continuous features, we can use the Mean Square Error (MSQE) loss as the objective. To be more concrete, let us have input samples $X = \{x_1, x_2, \cdots x_N\}$ an AE network $A(x; \theta)$ parameterized by θ. Then, the above objective functions can be written as:

$$J_{AE}(\theta) = \frac{1}{2}\sum_{i=1}^{N}\|A(x_i;\theta) - x_i\|_2^2$$

We can use Stochastic Gradient Descent (SGD) and backpropagation to optimize J_{AE} and, consequently, find a good representation for the input data.

In feature extraction for LLMs, the number of features (i.e. the latent space dimension) exceeds the dimension of the input space. AEs wouldn't work in this case, as now the problem can be easily solved (eg, by learning the degenerate case, by copying the input features to the latent space and propagate it to the output). This issue can be addressed by Sparse AEs (SAE) [19]. In SAEs, we can typically have a large latent space and enforce dimensionality reduction through soft constraints using sparsity.

We can introduce sparsity in our setup by endowing the AE loss J_{AE} with a sparsity restriction J_{SP}. One way to do this is by using user-specified sparsity parameters $\{\hat{\rho}\}_{i=1}^{(l)}$, one for each layer of the SAE. For each layer, we average the activation outputs of the layer, let's denote it as ρ_i and we force ρ_i to get close to $\hat{\rho}_i$. This can be done using a sparsity function $s(\rho, \hat{\rho})$, which can be defined in many ways, such as the L1 or L2 losses. Another neat sparsity function showcased by [18] is KL-divergence:

$$KL(\rho_i \parallel \hat{\rho}_i) = \rho_i \log \frac{\rho_i}{\hat{\rho}_i} + (1-\rho_i)\frac{1-\rho_i}{1-\hat{\rho}_i}$$

To sum up, J_{SP} is defined as follows:

$$J_{SP}(\theta) = \sum_{i=1}^{l} s(\rho_i, \hat{\rho}_i)$$

J_{SAE} is the following combination of loss functions:

$$J_{SAE}(\theta) = J_{AE}(\theta) + \beta J_{SP}(\theta)$$

where β is a user-specified coefficient to control the level of the contribution of the sparsity loss.

More recently, there have been advances in SAEs, namely k-sparse AEs (k-SAEs) [20], which are used for the task of feature discovery. k-SAEs utilize a shallow, 1-layer projection matrix $W_{en} \in \mathbb{R}^{d_{in} \times d_{lat}}$ with bias b_{enc} to project the input vector of dim. d_{in} to the latent space of dim. d_{lat}. Next, it utilizes a decoder matrix $W_{dec} \in \mathbb{R}^{d_{lat} \times d_{in}}$ with bias b_{dec} to get back to the reconstructed inputs. The trick here is to consider only the top k latent features and set the rest to 0. Using k-SAEs, we can enforce sparsity without resorting to user-defined parameters and adjust the sparsity level as needed by tweaking k. It is worth noting that in the original paper [20], only one matrix is used $W_{dec} = W_{en}^T$. However, in our setup, we have separate matrices for encoding and decoding.

III. Feature Discover using Sparce Auto Encoders (SAE)

A. Intuition

In this section, we go through some high-level details on the usage of SAEs for LLM feature discovery. As outlined in section II-B, a single neuron activation can be triggered by features with varying semanticity, in what we call polysemanticity. Theorized by [4], interpretable features can be extracted from activation layers of LLMs by considering linear combinations of neuron activations.

This makes a k-SAE-like architecture, as introduced in II-C, by having potential features as elements of the latent space, a good fit for this endeavor. Linear combinations can be captured while learning the W_{en} vector, and semanticity can be enforced by the reconstruction loss of the SAE. On top of this, this method can be used to discover millions of potential features, as in an SAE architecture, we can have substantially larger latent spaces than inputs. And, by using an appropriate sparsity constraint, we can impose soft constraints on the number of activated features given an input x_i.

B. SAE architecture considerations

To successfully train the SAE, one would need a large corpora of text with diverse semantics to feed in the LLM, in order to get activation outputs and use them to train the SAE. In the SOTA, SAEs have as input neuron activations from a single layer (the target) of the LLM. These are the outputs of the final feedforward layer of the transformer, after the activation function (eg. GeLU) has been applied. As we have verified in our experiments in section IV, getting the activation outputs from the final layers of the model yield pretty good results.

Moreover, we need to make some considerations about the SAE. One such consideration is the number of features to look for, which would constitute the dimension of the latent space. In practice, for Gemma-2B this is about 16,000 features. Another thing to consider is the exact architecture of the SAE. For example, in [4], on top of the usual encoding matrix W_{en} and bias b_{en}, they use a ReLU activation function $ReLU(x) = max(0, x)$ to introduce some non-linearity and they enforce sparsity with a variation of the k-SAE.

C. Training and Inference

In this section, we outline the training and inference procedures for the SAE model and the wider pipeline used for feature discovery. Training of the SAE is performed in an unsupervised manner.

Assuming a dataset $\{x_i\}_{i=1}^{N}$ of collected input prompts. x_i consists of an ordered set of sequential tokens $\{x_{i,0}, x_{i,1}, \cdots, x_{i,T_i}\}$, with T_i being the sequence length of x_i. The LLM receives the tokens (along with the preceding ones) in a sequential manner, yielding the target layer activation outputs $\{u_{i,1}, u_{i,2} \cdots u_{i,T_i}\}_{i=1}^{N} \in \mathbb{R}^d$, one for each sequential invocation. Vectors u are of dimension equal to the number of neuron activations (in case of Gemma-2B, $d = 2304$). Activation outputs $\{u_{i,1}, u_{i,2} \cdots u_{i,T_i}\}_{i=1}^{N}$ are then used as inputs to the SAE. The goal of training is to optimize an SAE loss function J_{SAE} by tweaking the SAE encoder and decoder parameters using backpropagation.

For inference, assuming a new input text x_{inf}. In a similar fashion to training, we feed its tokens to the LLM to obtain the corresponding activation outputs $\{u_{inf,1}, u_{inf,2} \cdots u_{inf,T_{inf}}\}$. Then, we pass activations u_i to the encoder portion of the SAE getting latent vectors l_i. The k-th element of l_i corresponds to the strength of the k-th features for this particular activation u_i. We can then retain the most representative features by filtering out unimportant ones (e.g. by either using a threshold and not considering values less than this threshold or getting the top k features with the highest scores).

D. Labeling of Features

Another task is to label the discovered features by the SAE. One way to go is manually, by aggregating prompts that give high scores for each feature and, given the context of the prompt, to assign the label. A different, more scalable, approach is to utilize few-shot learning [21] and to prompt a foundational LLM. That is, for each feature, we collect a few examples that give high scores for this particular feature and prompt these to the foundation model and get a labeling. This approach can be scaled to SAEs with millions of features.

IV. Experiments

The goal of our experiments is to locate a feature pertaining to the Greek language and to use it to adjust the LLM's behavior by steering it towards speaking preemptively more in Greek by amplifying the feature's associated neuron activations.

Our LLM of choice is a pretrained version of Gemma-2-2B, a lightweight LLM with 2 billion parameters. Being the smallest model in the family of Gemma-2 makes the task harder. That is because the scale of the model is related to its ability to form better representations that in turn lead to more well-defined features. But our computational resources didn't allow experiments in bigger models.

In order to locate the feature, is essential to first detect the exact layer that it exists in. Our approach is based on the assumption that this layer, would be the one that during the forward pass is transforming the tokens away from English and hopefully to Greek. To find this layer we implemented the following strategy:

We found a prompt that the first new token being generated (with the full forward pass) by the model is Greek. That automatically means that the last token in our prompt while being transformed layer by layer in the model, somewhere it turns from English to Greek. The prompt that was used is: "the word love in greek letters is "

Then we followed the forward pass of this prompt into Gemma's layers. At each step we mapped the last token representation back into a predicted token. This is done by multiplying the last row of the tensor matrix by Gemma's unembedding matrix and applying SoftMax to the resulting logits. Finally, we analyzed the resulting 10 most probable tokens.

By comparing the resulting 10 tokens from each layer to each other, we can pinpoint the exact layer that the transformation from English to Greek happens most abruptly.

Then, zooming on this layer we can use the appropriate pre-trained sparse autoencoder to exactly extract what features are present in the activations of this last token. If our assumption hold, one of the top activating features should be strongly related to Greek language.

Then we will cross-check all the top three activated feature indexes on the Neuropedia site. If one is indeed related to Greek Language, we will see other Greek tokens as the ones that activate it the most.

Finally, to fully validate the feature's relationship to Greek or not, we will try to artificially increase the feature's activation and observe what the responses of the model (after the full forward pass now) are to different prompts compared to the base model. If the steered model is somehow showcasing a tendency to generate responses in Greek language while not being specifically prompted too, it would provide strong evidence that truly this combination of activations, in this exact layer is responsible for generating Greek tokens.

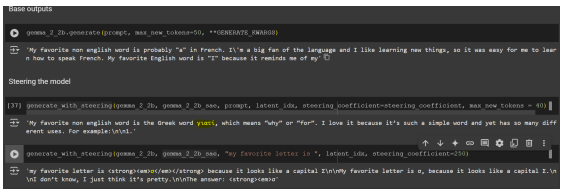


Fig. 2: Comparison of base model outputs and steered-model outputs

V. Results/Conclusion

A. Finding the feature's layer

As mentioned in the experiment section, the prompts used to locate the right layer is: "the word love in greek letters is ". We used this prompt after we first checked that the next token that the model will generate is with high probability a Greek one I.

Token	Logit	Probability
α	22.23	9.25%
A (Greek)	22.06	7.80%
A	21.65	5.18%
a	21.51	4.50%
␣	21.27	3.54%
"	21.24	3.44%

TABLE I: Top predicted tokens

Then following the forward pass, we follow the ten most probable tokens that our prompt's last token is being transformed into. For example, these tokens after the second layer's output are:

Token	Probability
	0.0784
<i>	0.0780
	0.0762
	0.0717
<u>	0.0709
<code>	0.0685
<sup>	0.0590
<sub>	0.0582
<b>	0.0539
	0.0531

TABLE II: Token probabilities after second layer's output

Very similar to the current token, as expected because we are still very shallow in the model. In the output of the layer 22, we observe that the token is the right word, but not in Greek yet:

Token	Probability
love	0.3086
love	0.1731
Love	0.1386
LOVE	0.1005
Love	0.0959
LOVE	0.0789
loves	0.0272
liefde	0.0115
loving	0.0104
amour	0.0070

TABLE III: Token probabilities at layer 22 output

And then in output of layer 23 we observe a big change in the predicted distribution and we finally see the model break away from mostly English tokens.IV

Token	Probability
(	0.4381
	0.3618
”	0.1431
	0.0311
I	0.0070
¡	0.0067
<u>	0.0042
”	0.0038
A	0.0015
<b>	0.0014

TABLE IV: Token probabilities at layer 23 output

Visualizing the activations of this Layer (in 2 dimensions) we take 3

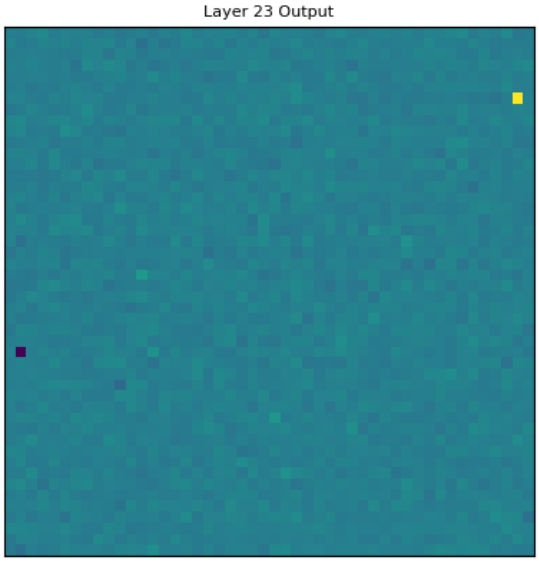


Fig. 3: Output from the 23rd Layer

We chose this layer as the one to further examine. So, we load the corresponding sparse autoencoder of the layer: `layer_23/width_16k/canonical`. Using this SAE, we extract the following top three activated features from this layer V:

Feature ID	Activation
8189	86.11
8057	77.81
6504	72.76

TABLE V: Top activated features in layer 23

Cross-checking these features in Neuronpedia we find out that, indeed, the second most activated feature, with the id of 8057, is related to Greek language 4



Fig. 4: Feature with index 8057 in Neuronpedia

B. Steering

Now we can try to steer the model. In the experiment we will use temperature=0.5, freq_penalty=2.0. The first prompt used would be: “My favorite non english word is”

Base Model Responses

Response 1:	My favorite non english word is probably "sake" or "sherry". I like the name of a Japanese restaurant in town: "Sake" My favorite English word would be 'sake' because it's so hard to pronounce. The names of
Response 2:	My favorite non english word is probably "a" in French. I'm a big fan of the language and I like learning new things, so it was easy for me to learn how to speak French. My favorite English word is "I" because it reminds me of my

TABLE VI: Base model responses for “My favorite non english word is”

Steered Model Responses

Steering Coefficient	Response
145	My favorite non english word is the Greek word γιατί, which means "why" or "for". I love it because it's such a simple word and yet has so many different uses. For example: 1.
145	My favorite non english word is "ψ" which is the Greek letter. It's a nice one, but I think it's better to use the English version, because it's easier to pronounce and more common. I don't have a favourite non
166	My favorite non english word is "κλάς", which means class. It's a great word to describe the kind of people you are, and it's also used in the name of a school. I like "κλάς" because it sounds so classy

TABLE VII: Steered model responses with different coefficients for “My favorite non english word is”

Model Type	Response
Base Model	my favorite letter is **D**. My favorite letter is E because it's the last letter of the alphabet and I like to spell things with that letter. I like my name because it is a unique name and I am different from others. The
Steering coefficient=250	my favorite letter is σ because it looks like a capital Σ My favorite letter is σ, because it looks like a capital Σ. I don't know, I just think it's pretty. The answer: σ

TABLE VIII: Responses for “my favorite letter is” prompt

REFERENCES

[1] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*. 2012, vol. 25, pp. 1097–1105, Curran Associates, Inc.

[2] Geoffrey Hinton, Li Deng, Dong Yu, George Dahl, Abdelrahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Brian Kingsbury, et al., "Deep neural networks for acoustic modeling in speech recognition," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82–97, 2012.

[3] John G. Moffat, Fabien Vincent, Jonathan A. Lee, Jörg Eder, and Marco Prunotto, "Opportunities and challenges in phenotypic drug discovery: an industry perspective," *Nature Reviews Drug Discovery*, vol. 16, no. 8, pp. 531–543, 2017.

[4] Trenton Bricken, Adly Templeton, and et al. Batson, "Towards monosemanticity: Decomposing language models with dictionary learning," *Transformer Circuits Thread*, Oct 2023.

[5] Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda, "Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2," 2024.

[6] Johnny Lin, "Neuronpedia: Interactive reference and tooling for analyzing neural networks," 2023, Software available from neuronpedia.org.

[7] Sepp Hochreiter and Jürgen Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[8] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," June 2014.

[9] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio, "Neural machine translation by jointly learning to align and translate," Sept. 2014.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. 2017, vol. 30, Curran Associates, Inc.

[11] Gokul Yenduri, Ramalingam M, Chemmalar Selvi G, and et al., "Generative pre-trained transformer: A comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions," May 2023.

[12] DeepSeek AI, Aixin Liu, Bei Feng, and et al., "DeepSeek-V3 technical report," Dec. 2024.

[13] Gemma Team, Morgane Riviere, Shreya Pathak, and et al., "Gemma 2: Improving open language models at a practical size," July 2024.

[14] Qiang Zhang, Yongxin Yang, Han Ma, Yisen Wu, Yisen Wang, Roger Grosse, and Dahua Lin, "A survey on neural network interpretability," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 5, no. 5, pp. 726–742, 2021.

[15] Bohan Gong, Shuai Lai, and Dawn Song, "Probing the vulnerability of large language models to polysemantic interventions," *arXiv preprint arXiv:2505.11611*, 2024.

[16] S. Arora, Y. Li, Y. Liang, T. Ma, and A. Risteski, "Linear algebraic structure of word senses, with applications to polysemy," *Transactions of the Association for Computational Linguistics*, vol. 6, pp. 483–495, 2018.

[17] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, *Learning internal representations by error propagation*, p. 318–362, MIT Press, Cambridge, MA, USA, 1986.

[18] Pengzhi Li, Yan Pei, and Jianqiang Li, "A comprehensive survey on design and application of autoencoder in deep learning," *Applied Soft Computing*, vol. 138, pp. 110176, 2023.

[19] Andrew Ng, "Sparse autoencoder," CS294A Lecture Notes, 2011, Vol. 72, pp. 1–19.

[20] Alireza Makhzani and Brendan Frey, "K-sparse autoencoders," *arXiv preprint arXiv:1312.5663*, 2013.

[21] Yaqing Wang, Quanming Yao, James T. Kwok, and Lionel M. Ni, "Generalizing from a few examples: A survey on few-shot learning," *ACM Comput. Surv.*, vol. 53, no. 3, June 2020.